

30 Matrix Common elements in all rows of a given matrix

```
// A Program to prints common element in all
// rows of matrix
#include <bits/stdc++.h>
using namespace std;
// Specify number of rows and columns
#define M 4
#define N 5
// prints common element in all rows of matrix
void printCommonElements(int mat[M][N])
{
    unordered_map<int, int> mp;
    // initialize 1st row elements with value 1
    for (int j = 0; j < N; j++)
        mp[(mat[0][j])] = 1;
    // traverse the matrix
    for (int i = 1; i < M; i++)
        for (int j = 0; j < N; j++)
        {
            // If element is present in the map and
            // is not duplicated in current row.
            if (mp[(mat[i][j])] == i)
            {
                // we increment count of the element
                // in map by 1
                mp[(mat[i][j])] = i + 1;

                // If this is last row
                if (i == M - 1 && mp[(mat[i][j])] == M)
                    cout << mat[i][j] << " ";
            }
        }
    cout << endl;
}

// driver program to test above function
int main()
{
    int mat[M][N] =
    {
        {1, 2, 1, 4, 8},
        {3, 7, 8, 5, 1},
        {8, 7, 7, 3, 1},
        {8, 1, 2, 7, 9},
    };
    printCommonElements(mat);
    return 0;
}
```



The screenshot shows a C++ program being executed in a terminal window. The program defines a 4x5 matrix and a function to find common elements in all rows. The output of the program is '8 1', which are the common elements in all rows of the matrix. Below the terminal window, the text 'Output' is displayed, followed by the same output '8 1'. At the bottom, a note states: 'The time complexity of this solution is $O(m * n)$ and we are doing only one traversal of the matrix. Auxiliary Space: $O(N)$ '.

```
// driver program to test above function
int main()
{
    int mat[M][N] =
    {
        {1, 2, 1, 4, 8},
        {3, 7, 8, 5, 1},
        {8, 7, 7, 3, 1},
        {8, 1, 2, 7, 9},
    };
    printCommonElements(mat);
    return 0;
}
```

Output

8 1

The time complexity of this solution is $O(m * n)$ and we are doing only one traversal of the matrix.
Auxiliary Space: $O(N)$